

What I did not know when I started

Three things I did not know when I opened the packet — one about the regulation, one about method, one about design. For each: the situation, the gap, how I closed it, and how far I can now go with it.

1. SR 26-2 ITSELF

Situation. The bank's triggers are SR 26-2 and DORA. The brief has to say what changed and why it matters to a Chief Audit Executive, in under 250 words, every sentence sourced.

What I did not know. Whether SR 26-2 existed — the packet is "messy on purpose" — or, if it did, what it said. My first plan was to run DORA and avoid it.

How I learned it. A search found the Fed's page: issued 17 April 2026 by the Fed, FDIC and OCC, replacing the 2011 and 2021 model-risk guidance, aimed at banks over \$30 billion. The summary page says nothing about AI. The 12-page attachment does, in a footnote: generative and agentic AI models are treated as too new to be covered, so the guidance's principles apply only to traditional and non-agentic models, and the bank's own risk management is left to decide the controls for the rest. The obvious pitch — "AI governance just tightened, you need assurance" — is the opposite of what the document says.

How I got dangerous. I can now say to a CAE what the letter does and does not cover, that the supervisors have formally named the gap around agents rather than closed it, that the attachment was touched once since publication (4 May — a single corrected word), and which of the letters that followed it — SR 26-3 through 26-6 — have nothing to do with model risk. That is the brief's core sentence, and it is the reason the agent's test verdict on SR 26-6 was "noise" rather than a false brief.

2. TESTING A WATCHER WHEN THE WORLD DOES NOT CHANGE

Situation. The system exists to notice regulators changing something. Nothing had changed and nothing would on my schedule.

What I did not know. Where past versions of a Fed page or an EU regulation live, how to fetch them in a form my tools could compare against today's page, and whether what I found would be a real event or an archive artifact.

How I learned it. `Last-Modified` headers showed the SR 26-2 files touched on 4 May and the index page changed two days before I started watching. The Wayback Machine fought back at each step — rate limits, an HTML wrapper where the PDF should be, gzip-compressed captures — until the timemap endpoint and the `id_` flag gave me every

capture as raw bytes. A first fixture built the wrong way carried rewritten links that all looked new; the agent read the diff, saw the text was unchanged, and called it noise with the reason spelled out.

How I got dangerous. I can reconstruct the change history of any public regulatory document — what changed, when, by how much — and turn any moment of it into a repeatable test. That became the test mode: two scenarios that seed baselines from real captures in a separate state tree, so a check replays what actually happened. Scenario A produced a brief per affected account; scenario B a noise verdict with the diff kept.

3. MAKING AN AGENT OBEY A RULE, NOT JUST READ IT

Situation. R-17: any action touching a financial-services account writes an audit record first, or the action does not happen.

What I did not know. How to make that true for an LLM. My first instinct was to write "call `write_audit` first" into the skill.

How I learned it. By noticing that an instruction is a request the model can skip under pressure, and that nothing would tell me if it had. So the rule moved into code: `emit_brief` is the only path from draft to file, and inside it an `@audited` decorator makes the render unreachable until `write_audit` has returned. `write_audit` itself refuses any record with a missing field, a vague purpose, or an unresolvable person. A seventh eval check then verifies, independently, that the record exists and predates the brief — so a brief produced any other way fails and never gets a hand-off.

How I got dangerous. I can now build agent tooling that fails closed: the model decides *what*, the code decides *whether*, and a third party can check both. The same shape is on the roadmap for the send step — an `@audited` `send_message` that writes the R-17 `message` record, checks the approval, and only then does anything.